QUITENIGHT

Assignment 3

Hina Iqbal
2023182290

Quiet Night: IoT-Based Remote Monitoring System Using ESP32

1. Introduction

The "Quiet Night" project is a compact IoT solution designed to remotely monitor environmental and activity-based parameters through an integrated system of sensors, communication protocols, and a cloud-powered interface. It uses the ESP32 microcontroller with sensors to gather data on temperature, humidity, light intensity, and vibrations. Data is relayed via MQTT and stored on Google Firebase, while a Streamlit-based web UI provides real-time visualization and alerts. This project showcases the feasibility of scalable, user-friendly, and secure remote monitoring.

2. System Overview and Objectives

The objective is to create a secure, real-time sensing system capable of scaling to multiple nodes. The ESP32 collects data from a DHT22 (temperature and humidity), a TSL2561 (light), and a piezoelectric vibration sensor. These readings are sent to a cloud database via MQTT. The system decouples sensor data from storage using a Node.js cloud function that listens to MQTT messages and uploads them to Firebase, thus enhancing stability and security. The project demonstrates a reliable pipeline from sensor data acquisition to user-friendly output.

3. Hardware and Sensor Integration

The ESP32 is connected to the DHT22 for measuring temperature and humidity, the TSL2561 for detecting visible and IR light, and a piezoelectric sensor for vibration detection. These sensors are interfaced via GPIO pins, and data is collected periodically. This simple but robust hardware setup allows real-time tracking of environmental variables.

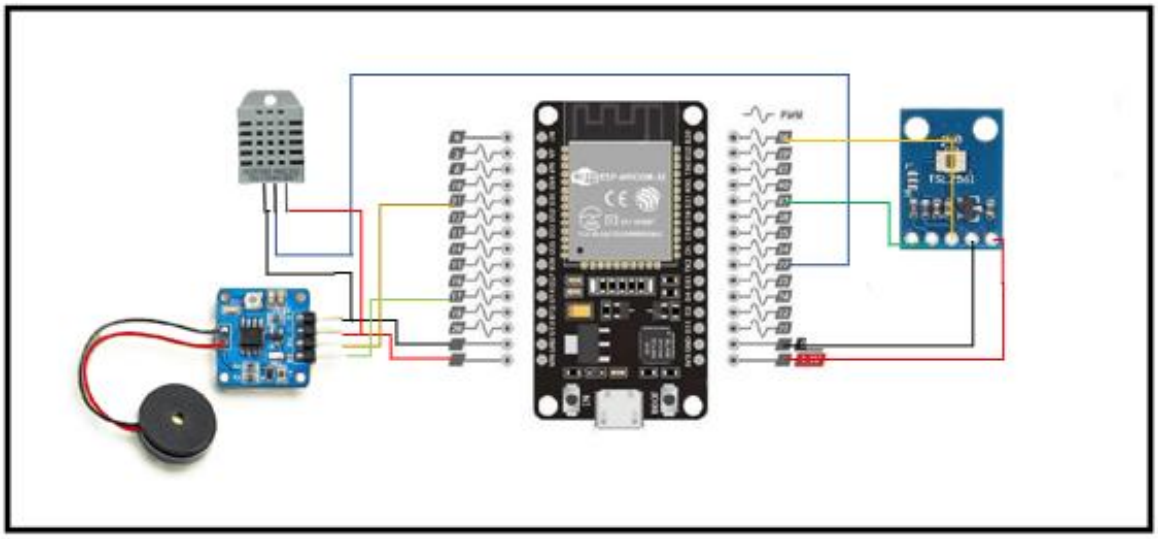


Figure 1: Connection diagram of IoT system

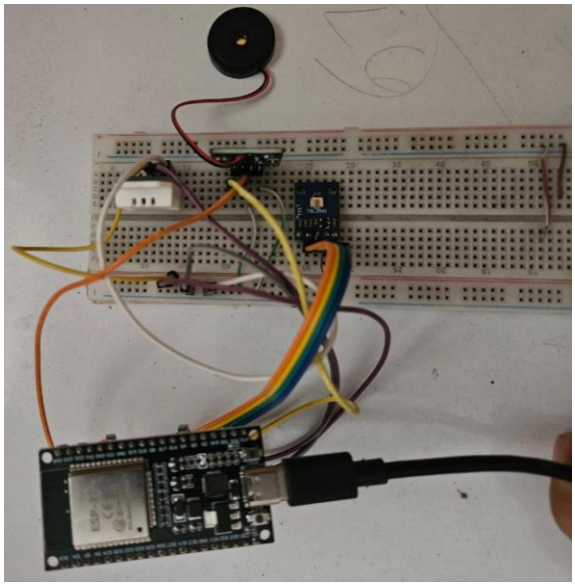


Figure 2: Physical picture of the IoT system

The connection diagram illustrates the sensor-ESP32 wiring, and the physical image shows the assembled device.

4. Communication and Data Flow

Data from the ESP32 is sent to the MQTT broker under the topic `quietnight/data/esp32`. A Node.js cloud function, subscribed to this topic, captures and relays the data to the Firebase Realtime Database. This intermediate cloud layer prevents direct access from devices to the database, enhancing both data security and integrity. Firebase holds readings under two nodes—physical and simulated.

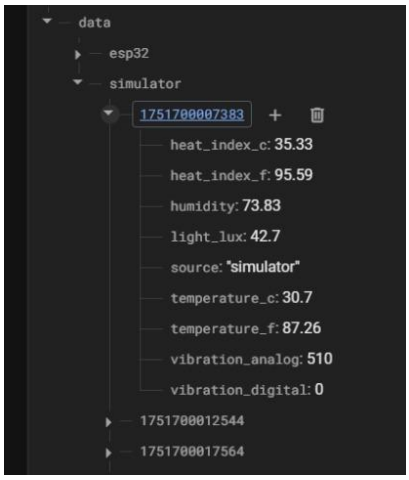


Figure 3: Firebase Realtime Database showing two nodes and their readings

5. Software Components and User Interface

ESP32 firmware was written in C++ using the Arduino IDE, which handles sensor data collection and MQTT publishing. The Node.js cloud function uploads this data to Firebase. The frontend UI, built in Streamlit, has a login page, a dashboard for live sensor readings, a graph-based visualization page, and a threshold alert system. Alerts notify users when a reading crosses a predefined limit.

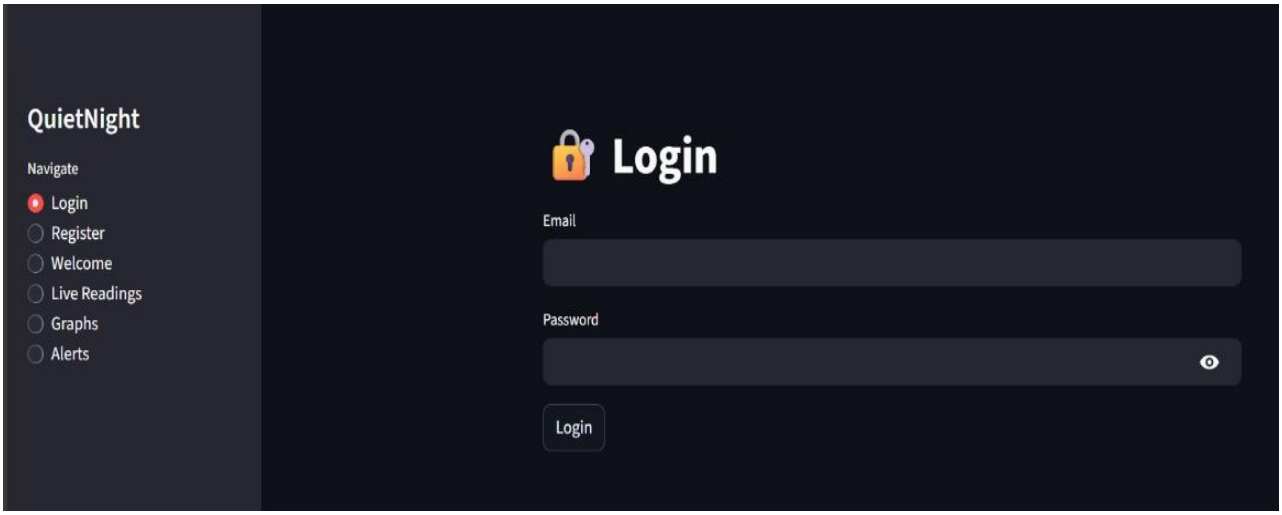


Figure 3: Login page of Streamlit UI



Figure 4: Live numerical readings in Streamlit

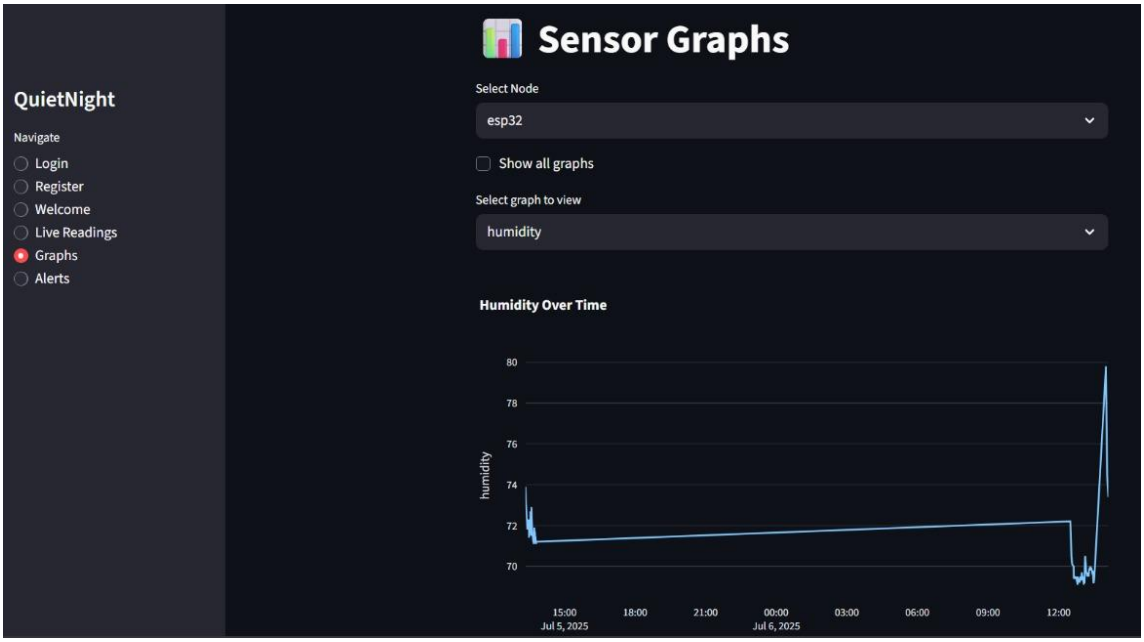


Figure 6: Graph visualization over time

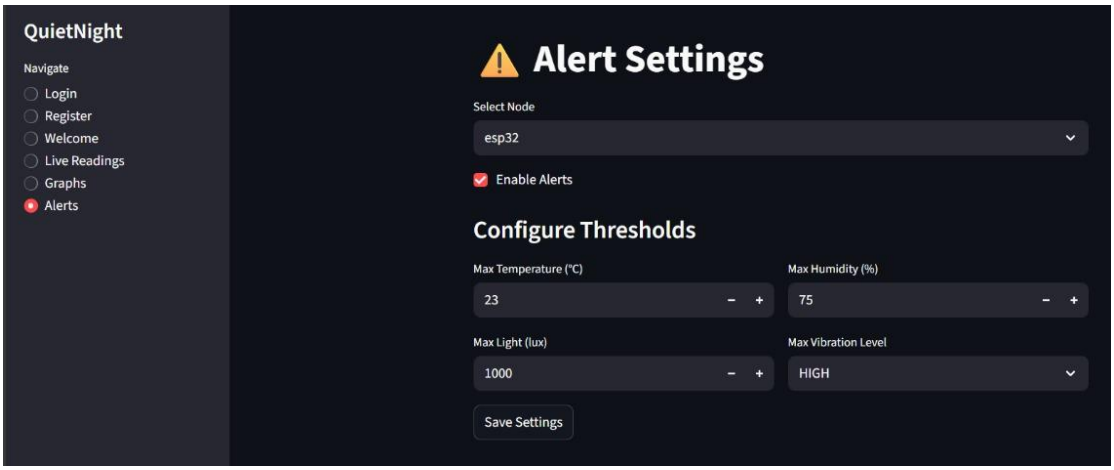


Figure 7: Alert settings page

These interface pages allow users to monitor and interact with the system seamlessly.

6. Scalability and Simulated Node Testing

To prove scalability, a simulated node was built in Google Colab that generates and sends random sensor data to the MQTT topic. The cloud function processes this like real data, creating a second node in Firebase. The UI treats both nodes independently, demonstrating support for multiple sensor units with no backend changes.

7. Operational Challenges and Bi-directional Communication

Wi-Fi instability during testing was resolved with reconnect logic in the ESP32 firmware. Timestamp management was handled via onboard time or backend functions. A basic form of bi-directional communication was enabled, where ESP32 could pull values from Firebase, paving the way for cloud-to-device commands in future upgrades.

8. Usefulness and Practical Applications

Real-time monitoring becomes significantly more useful with visual dashboards and alerts. The UI allows users to observe trends and act on insights rather than raw numbers. Potential applications include home security, greenhouse monitoring, or machinery diagnostics. The modular build enables easy sensor or feature expansion.

9. Components and Technologies Used

Category	Component/Technology
Microcontroller	ESP32
Sensors	DHT22, TSL2561, Piezoelectric Vibration Sensor
Communication	MQTT (Mosquitto)
Cloud Backend	Firebase Realtime Database
Middleware	Node.js Cloud Function
Frontend	Streamlit (Python)
Firmware	Arduino IDE (C++)
Simulation	Google Colab (Python)

10. Conclusion

The "Quiet Night" system successfully combines IoT hardware, reliable communication, and interactive UI features into a real-time monitoring platform. It ensures data safety, scalability, and usability, providing an adaptable solution for various smart monitoring applications. Its strength lies in its layered design and modularity, making it ideal for further development and deployment.